Radar Tracking Software Pipeline Optimisation for UAV Detect-and-Avoid

NOAH WEST

B.Eng (Hons)

Supervisor: Dr. Donald Dansereau

A thesis submitted in fulfilment of
the requirements for the degree of
Bachelor of Engineering (Honours)

School of Aerospace Mechanical and Mechatronics Engineering
Faculty of Engineering
The University of Sydney
Australia

27 March 2026

Abstract

- Rodar is able to provide Unmanned Aeriel Vehicales with the capability to detect and track airborne hazzards over large distances and in all weather conditions. Talk about trade off between size, power, weight and compute. Talk about problems.
- We seek to address these problems through adapting establish radar processing algorithms for UAVs.
- By evaluating the effectiveness of different implementations and algorithms essential to multi target tracking, this report demonstrates how software choises make long range radar practical for small UAV detect and avoid systems.
- Talk about key parts of our pipeline
- Talk about the Experiments
- Talk about findings
- Ultimately, our work provides a reference for engineers and policy makers...

Statement of Contributions

I, **Noah West**, declare that the intellectual content of this thesis is the product of my own work and that all assistance received in the preparation of this thesis and relevant sources have been appropriately acknowledged. The contributions I made to this research include:

- Undertaking the background research and completing the literature review.
- Defining the research scope and developing the problem statement in consultation with my supervisor and the industry partner.
- Designing the methodology, experimental approach, and test plan with guidance from my supervisor.
- Designing and implementing, in collaboration with engineers from the industry partner *Mission Systems*, the software pipeline described in this work.
- Profiling and evaluating the pipeline software to produce the reported results.
- Capturing radar data for use in testing and validation.
- Conducting the experiments, including configuration of the test setup and execution of data collection procedures.
- Performing analysis of results, including quantitative evaluation using appropriate metrics to validate conclusions.
- Interpreting findings, and writing the discussion and conclusions, incorporating feedback from my supervisor and relevant stakeholders.
- Writing the thesis and preparing figures, tables, and supporting material.

The above represents an accurate summary of my contribution.

Noah West
Honours Candidate
School of Aerospace, Mechanical and Mechatronic Engineering,
The University of Sydney
Date: 27th March 2026

Dr. Donald Dansereau
Senior Lecturer
Australian Centre for Field Robotics,
The University of Sydney
Date: . . . . . .

Acknowledgements

I would like to express my deepest gratitude to my supervisor, Dr. Donald Dansereau, for his guidance and support throughout this research.

I am also thankful to my industry mentors Rex, Hugh, and David Johnson. This work would not be possible without their enthusiastic support, and expertise.

Contents

Abstract ii
Statement of Contributions iii
Acknowledgements iv

Contents **v**

List of Figures **ix**

Chapter 1 Introduction **1**

Chapter 2 Background **6**
2.1 Radar Processing 6
2.2 Monopulse Radar Hardware 9
2.3 Multi Target Tracking 11
2.4 Parallel Processing 13

Chapter 3 Literature Review **15**
3.1 Compact Airborne Radar for UAV DAA 15
3.2 Embedded Radar Processing Under SWaP Constraints 16
3.3 Sensitivity and Detection in Low-SNR Regimes 17
3.4 Summary and Research Gap 17

Chapter 4 Methodology **19**
4.1 Overview 19
4.2 Software Design 19
4.2.1 Data Types and Protocols 21
4.2.2 Preprocessing Application 21
4.2.3 Aggregator Application 21

4.2.4 Detection and Tracking Application 21
4.3 Processing Application Evaluation 25
4.3.1 Windowing 25
4.3.2 Coherent Integration 26
4.4 Tracking Application Evaluation 26
4.4.1 CFAR 26
4.4.2 Monopulse Angle Sensing 30
4.5 Testing Datasets 31
4.6 Performance Analysis 31
4.6.1 Throughput Analysis 31
4.6.2 Latency Analysis 31
4.7 Computational Performance Metrics and Analysis Methods 32
4.8 Tracking Performance Metrics and Analysis Methods 33
4.9 Results 34
4.10 Processing Application 34
4.10.1 Windowing 34
4.10.2 Coherent Integration 34
4.11 Tracking Application 34
4.11.1 Constant False Alarm Rate 34
4.11.1.1 Correctness 34
4.11.1.2 Computational Performance 35
4.11.2 Monopulse Angle Sensing 36

Chapter 5 Discussion **39**
5.1 Processing Application 39
5.1.1 Windowing Operation 39
5.1.2 Coherent Integration Operation 39
5.2 Tracking Application 40
5.2.1 CFAR Operation 40
5.2.2 Monopulse Angle Sensing Operation 40
5.3 Addressing the Identified Knowledge Gaps 41

Chapter 6 Conclusion **43**
6.1 Summary of Findings 43
6.2 Industry Partner Outcomes 43
6.3 Future Work 44

Bibliography **46**

Appendix A Appendix **49**
A1 Case Studies 49
A2 Case Studies: Learning Objectives Covered by Thesis 49
A2.1 Case Study 1: AMME5520 Advanced Control and Optimisation 49
A2.1.1 LO1. *Approach research papers in a professional and research-orientated manner, and conduct critical reviews of these papers* 49
A2.1.2 LO2. *Implement simple path generation algorithms, controllers and decision metrics for an autonomous system in order to meet specific mission objectives* 50
A2.1.3 LO3. *Understand a number of different path generation and control algorithms implemented in autonomous systems and how they are linked to optimality criteria, platform stability and vehicle constraints.* 50
A2.1.4 LO4. *Understand how cost functions are used to define mission objectives in a mathematical form, so that autonomous systems can make decisions about their next action.* 51
A2.1.5 Coverage statement 51
A2.2 Case Study 2: ELEC3305 Digital Signal Processing 51
A2.2.1 LO1. *Describe signals using mathematical concepts such as convolutions, transforms, spectral analyses, linear difference equations, filters, correlation and covariance* 51
A2.2.2 LO2. *Use software, such as MATLAB and Python, to develop code to analyse and process signals including filtering, inverse filtering, spectral analyses, resampling, time-frequency analyses* 52

A2.2.3 LO3. *Identify linear signal models for time-invariant systems such as convolutional models, sinusoidal and harmonic models, echo and reverberation models, modulation models and time-frequency models.* 52
A2.2.4 LO4. *Design and evaluate signal processing systems.* 52
A2.2.5 Coverage statement 52
A3 Work Health and Safety Report 53
A3.1 Work, Health and Safety at Mission Systems 53
A3.2 Incidents and Preventions During Placement 53
A4 Practical Experience Logbook 53

List of Figures

2.1 The transmitted LFMCW chirp (top), delayed echo (middle), and resulting beat signal after mixing and low-pass filtering (bottom), shown for a close target (left, 0.5 µs delay) and far target (right, 2 µs delay). The far target has a higher beat frequency that the close target. 9

4.1 Application level dataflow. The pre-processing application receives the AWR2243's raw radar data over four CSI video data streams. The data is organised into frames. The processing application then outputs the data in the form of chirps containing target ranges. The aggregator application receives the four processed data streams, and combines them into a single data stream organised by quadruplet, a collection of four receivers necessary for monopulse angle sensing. The detection and tracking application receives a stream of quadruplets, and produces tracks of targets in the radar's field of view. 20

4.2 Dataflow for the preprocessing application. Multiple instances of the application run on separate processes, with each instance receiving compressed baseband radar data samples from an incoming video stream. Formatting organises samples into contiguous blocks in memory, and discards empty chirps. The windowing stage applies a window function to the data to reduce spectral smearing in the next state. The FFT stage applies a Fourier Transform across fast time into the frequency domain, revealing target range. The coherent integration stage sums the complex FFT values of identical pulses within the same frame. 22

4.3 Dataflow for the aggregator application. The application receives the four processed data streams from the preprocessing application, and combines them into a single data stream organised by quadruplet, a collection of four receivers necessary for monopulse angle sensing. 23

4.4 Dataflow for the detection and tracking application. The application receives a stream of quadruplets, and produces tracks of targets in the radar's field of view. By applying the following operations: CFAR, monopulse angle sensing, polar transform, data association, and tracking filters. 24
4.5 Results of an FFT performed on a signal with two frequencies and Gaussian noise, with and without a Hann window. Without the Hann window, the bins close to the peak frequency bins 40 and 45 are significantly larger than bins further away, misrepresenting the homogeneity of the Gaussian noise. 25
4.6 Coherent integration across five chirps for each range bin. Each arrow represents the complex FFT value of a single bin, with the real and imaginary components shown on the horizontal and vertical axes of each chirp panel. Most bins contain noise-like phasors with varying direction, so their coherent sum remains small. The highlighted bin contains a weak but consistent signal component across chirps, so its phasors add constructively, producing a larger resultant in the coherent-sum panel. 27
4.7 Naive Implementation of CFAR for the CPU. Each sample is compared to a threshold determined by the power of the samples around it. The threshold is calculated by taking the average power of the surrounding samples and multiplying it by a factor α determined by the desired false alarm rate. 29
4.8 CFAR implementation using a prefix sum table (cumulative signal power). The prefix sum table allows the noise statistic to be calculated with a fixed number of sums per sample of interest, regardless of the number of training cells. In this specific case, after the cumulative signal power is calculated in preprocessing, it only takes four sums of to find the sum of eight training cells. 29
4.9 CFAR implementation on the GPU. Threads (each depicted as a unique colours) are each responsible for a sample, allowing for parallel processing of the CFAR algorithm across the entire range profile. Each thread calculates its own noise statistic, threshold, and mask. 30
4.10 Results 35

4.11 CFAR with a desired false alarm rate of 0.001 infrequently detect peaks within a target less noise as expected. The figure is a waterfall plot, with chirp on the y-axis and range on the x-axis. The brightness of the cell reflects the power of the corresponding chirp's range bin. The black space indicates that most cells are masked by CFAR. 36
4.12 Computational Performance of three CFAR implementations as a function of buffer size on the desktop setup. The CPU implementation with a prefix sum tables is the fastest for small buffer sizes, but the GPU implementation is slightly faster for larger buffer sizes. The naive CPU implementation is the slowest across all buffer sizes. 37
4.13 Computational Performance of three CFAR implementations as a function of buffer size on the Xavier setup. The CPU implementation with a prefix sum tables is the fastest for small buffer sizes, but the GPU implementation is slightly faster for larger buffer sizes. The naive CPU implementation is the slowest across all buffer sizes. 37
4.14 Computational Performance of three CFAR implementations as a function of the number of training cells on the Desktop Setup. Buffer size is fixed at 10 quadruplets. 38
4.15 Computational Performance of three CFAR implementations as a function of the number of training cells on the Xavier Setup. Buffer size is fixed at 10 quadruplets. 38

CHAPTER 1

Introduction

With more uncrewed aerial vehicles (UAVs) in the air each year, and a growing desire for them to operate in controlled airspaces, there is a concern regarding their ability to maintain separation from hazards in the air. United States and European federal agencies are developing policy that necessitates detect-and-avoid (DAA) systems for UAVs, which parallel the visual and instrument flight rules (VFR & IFR) that dictate how human pilots avoid traffic [1] [2] [3].

While some proposed DAA systems utilise optical and infrared sensors [3], radar sensors may also be suitable, as they can perform sensing over great distances in all weather conditions, due to their relatively low atmospheric attenuation [4]. The inconvenient side effect of their long wavelength property is that angular resolution is much lower compared to similarly sized optical systems. If the challenges stemming from this limitation are overcome, the implementation of radar as the primary sensing mode or an additional sensing mode will improve the perception capabilities, and thus the safety, of UAVs.

Historically, airborne radar has been used for large aircraft, where size weight and power (SWaP) constraints are not strict. The size and weight of radar systems has been the largest barrier to their use on smaller platforms, [5], [6], [7]. This historical constraint is now weakening because both sides of the radar stack have miniaturised. Texas Instrument's AWR2243 is a single 76-81 GHz radar chip that integrates phase-locked loops (PLLs), three transmitters, four receivers, a mixer, an ADC, and self calibration, all on a 10.4mm $\times$ 10.4mm printed circuit board (PCB) -friendly package. Higher frequency, lower noise, electronics and the miniaturisation of the transistor have been key enablers of this technology. Higher frequency radar allows of higher resolution imaging or smaller aperture; lower noise allows

for an increase sensitivity to targets. On the compute side, modern UAV systems often have a dedicated, high compute companion computer in addition to the flight compute. NVIDIA's Jetson series computers such as Xavier NX, Orin Nano and Orin NX providing up 21, 67, 157 trillion operations per second (TOPS) on 8-bit integers respectively, while each weighting under 50g (without a heat sink).

Plextek's mmWave-based 60 GHz sense-and-avoid radar for UAVs demonstrates the feasibility of pairing miniaturised radar hardware with modern edge computing for the detection of people, vehicles, poles, and buildings at ranges of hundreds of metres. Echodyne's EchoFlight, a 817 g commercial radar systems for UAVs that can detect small drone targets at a range of 750 m, reflects the onset of the technology into industry [8]. The wider UAV market shows a similar shift toward more computationally demanding onboard tasks. Skydio's X10, for instance, incorporates both an NVIDIA Jetson Orin SoC and a Qualcomm QRB5165 SoC within a 2.11 kg platform while retaining up to 40 minutes of flight time, illustrating that substantial embedded compute is now compatible with practical UAV SWaP limits. These developments suggest that recent improvements in radar miniaturisation and embedded heterogeneous computing are promoting a new class of computation-heavy UAV sensing systems, making it timely to revisit radar target tracking and to determine the most suitable signal-processing pipeline for UAV detect-and-avoid applications.

Despite the challenges, UAVs are the fastest growing market for radar systems, projected to grow with a compound annual growth rate (CAGR) of 11.22% globally through 2030 [9]. To make these emerging radar systems more performant, accessible, and safe, it would be valuable to find the most suitable radar signal processing pipeline for UAV DAA systems, revisiting the old problem of radar target tracking in the face of unique hardware conditions.

Software-defined architecture (SDAs) are increasingly preferred over hardware defined radar systems for UAV applications as software modifications can be shipped faster than hardware modifications [9]. As a result, graphics processing units (GPUs) are favoured over non-programmable digital signal processors such as application specific integrated circuits (ASICs). GPU-accelerated implementations are also favourable for computationally intensive stages of the radar processing chain, as many core operations, such as fast Fourier transforms (FFTs)

and matrix operations, are highly data-parallel, allowing for large increases in throughput when compared to serial processors. Additionally, GPU-based pipelines can leverage mature developer friendly, GPU software ecosystems and optimised libraries [10]. Additionally, the incorporation of AI into radar tracking algorithms has further encouraged these highly parallel, GPU-based pipelines [11]. Key open questions remain around balancing the compute saved through GPU parallelism against the cost of data movement and synchronisation, and how unified (shared) CPU–GPU memory on embedded platforms changes this trade-off.

Given the miniaturisation of radar enabling technology, the potential benefits for radar base DAA systems, and the range limitations of systems suitable for small radar systems. This report proposes a software pipeline design that allows for the radar tracking of aerial hazards on UAVs with strict SWaP constraints. Particular emphasis will be placed on maximising the sensitivity to faraway targets, a requirement for operating in shared airspace, and minimising computation resources and latency.

The literature reviewed in Chapter 3 collectively demonstrates that compact radar hardware is now sufficiently mature for UAV DAA applications. SWaP-compliant systems have been built, detection at DAA-relevant ranges has been achieved, and processing feasibility on constrained embedded platforms has been established. However, three important gaps are identified.

First, existing hardware demonstrations validate detection only against large, high radar cross-section targets such as buildings and fixed-wing aircraft, leaving the practical sensitivity floor of compact UAV radar systems against small moving airborne targets unknown.

Second, no work has benchmarked a GPU-based radar pipeline in a UAV DAA setting. Existing studies are limited to FPGA and ARM CPU implementations, evaluate single algorithm sequences on single hardware configurations, and do not profile architectural bottlenecks such as host–device data transfer and thread synchronisation overhead.

Third, sensitivity algorithms for low-SNR monopulse radar have only been validated under idealised white Gaussian noise with single simulated targets, and none have been integrated into a complete end-to-end range–Doppler tracking pipeline.

The core problem this thesis addresses is the absence of a principled, hardware-aware understanding of how algorithm selection and implementation choices jointly affect the sensitivity and computational performance of compact monopulse radar systems for UAV DAA. This thesis constructs that understanding through a systems-level investigation of the software design space of a GPU-based radar tracking pipeline, with specific contributions as follows:

- A comparison of candidate detection, angle-estimation, and tracking algorithm combinations, assessed by detection sensitivity and track quality in multi-target aerial scenarios, to inform algorithm selection.
- A comparison of alternative low-level implementations of the selected algorithms, quantifying their effects on end-to-end throughput and latency in highly parallelised GPU pipelines, to inform implementation choices.
- An assessment of how these algorithm and implementation conclusions vary across embedded hardware configurations, providing hardware-aware design recommendations.

This thesis concerns the software design and evaluation of radar tracking pipelines for UAV DAA under SWaP constraints. The scope is limited to processing stages occurring after radar sample acquisition, with emphasis on how alternative signal-processing, angle-estimation, and tracking algorithms perform on embedded compute platforms suitable for small and medium UAVs. The processing chain considered spans front-end windowing, FFT, coherent integration, CFAR detection, monopulse angle estimation, coordinate transformation, data association, tracking filters, and track management, together with orchestration decisions such as batching, host–device transfers, and synchronisation overhead.

Several topics are outside scope. Radar front-end hardware design, full-scale field trials, FPGA and ASIC implementations, and cross-modality comparisons with optical or infrared sensing are not addressed. The thesis does not claim certification readiness for operational deployment; rather, it provides comparative evidence and design guidance for onboard radar processing under realistic embedded constraints.

The thesis proceeds under three assumptions. The radar sensor is a small conformal monopulse radar whose aperture and power budget are representative of a UAV-compatible installation. The target compute platform is GPU-enabled embedded hardware, reflecting the increasing use of heterogeneous CPU/GPU companion computers in modern UAV systems. The hardware context of primary interest features shared or unified CPU/GPU memory, as found in many embedded systems-on-module, though comparison against CPU-only and discrete-memory configurations is used where available to clarify architectural trade-offs.

CHAPTER 2

Background

2.1 Radar Processing

- Radar processing converts received echoes into target measurements such as range, velocity, and angle.
- Radar hardware performs transmission, reception, amplification, pulse-compression, downconversion, and analogue-to-digital conversion, while software extracts useful target information from sampled data.
- In software, a typical processing chain begins by organising the pulse-compressed samples across fast time, slow time, and antenna channels.
- By considering the samples the spectral domain (either with matched filtering or a fast time Fourier Transform), we can estimate the range of targets.
- Optionally, a long time Fourier transform can be applied across slow time to estimate Doppler, which provides target radial velocity and helps separate moving targets from clutter.
- Coherent integration across multiple pulses improves detectability of weak targets. This involves summing the complex samples across pulses.
- After integration detection algorithms such as Constant False Alarm Rate (CFAR) can identify candidate target returns, which are then passed to angle estimation and tracking stages.
- The bearing and elevation of targets can be estimated using monopulse processing, which compares the received signal across multiple antenna channels to infer angle with high accuracy.

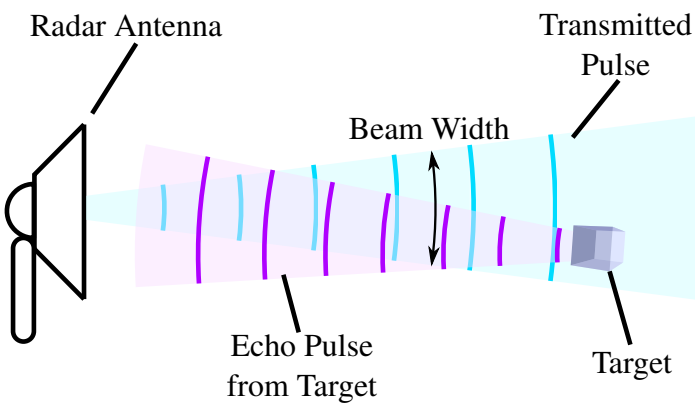
- The resulting range, Doppler, and angle measurements form the basis for later tracking and classification stages in a DAA system.

Every radar system operates by transmitting electromagnetic waves into an environment and receiving the reflected signals to detect and locate targets, (Figure 2.1). The strength of the received signal is given by the equation,

$$P_r = \frac{P_t G_t G_r \lambda^2 \sigma}{(4\pi)^3 R^4 L} \tag{2.1}$$

where P_r is the received power, P_t is the transmitted power, G_t and G_r are the gains of the transmitting and receiving antennas, λ is the wavelength, σ is the radar cross-section of the target, R is the range to the target, and L represents system losses.

Hence, the received signal informs us about the target's radar cross-section, reflectivity and range, while the time delay between transmission and reception gives us just range. The Doppler shift of the received signal provides information about the target's radial velocity. If there are multiple receiving channels, the relative phase and amplitude of the received signals across channels can be used to estimate the bearing and elevation of the target.

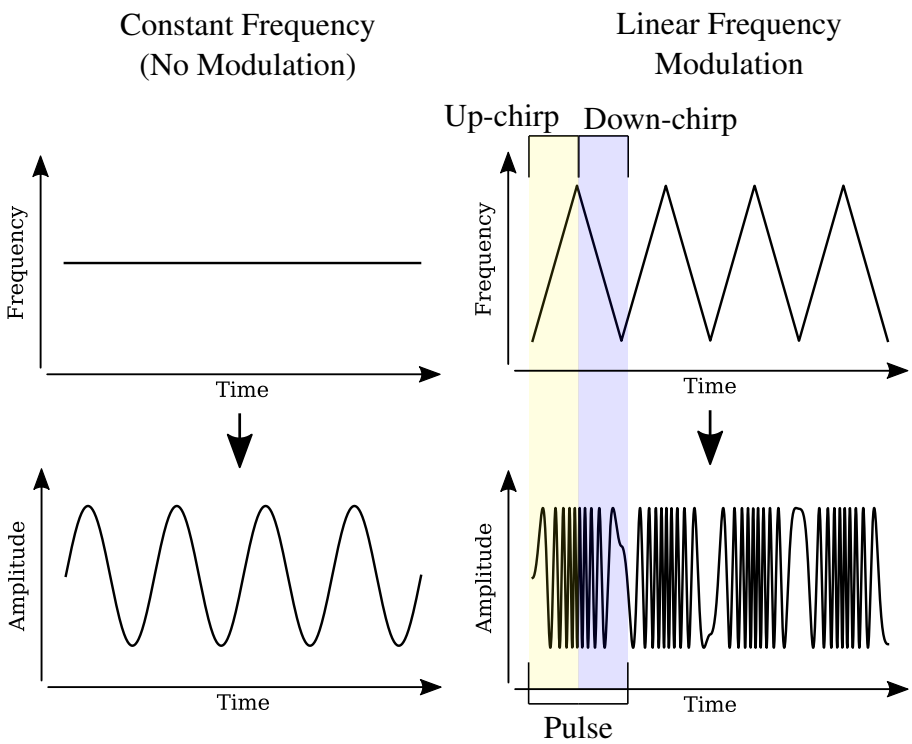


To measure the range of targets, radar systems need to transmit the signal as pulses. Pulses are generally preferred over continuous constant frequency signal because the time difference between the transmitted pulse and the received echo can be easily measured to estimate range. The transmitted signal is often modulated in frequency or phase. Linearly frequency modulation, (LFM) creates pulses that involve a continuous sweep of frequencies during the

pulse duration (see Figure 2.1). Advantages of such a modulation include reduced sensitivity to frequency dependent interference, and improved range resolution due to the effect of pulse compression. Through pulse compression, the long duration LFM pulse can be compressed in time to illustrate the difference between the transmitted and received signals. One pulse compression technique for LFMCW signals is described as "de-chirping", which involves mixing (multiplying) the received signal with the transmitted signal to produce a "beat" frequency that is proportional to the range of the target (See Figure 2.1). The resulting signal's frequencies are given by the product of cosines identity,

$$\cos A \cos B = \frac{1}{2}[\cos(A - B) + \cos(A + B)] \tag{2.2}$$

where A is the transmitted signal and B is the received signal. By applying a low-pass filter to the mixed signal, we obtain the lower frequency component, which is the beat frequency. The beat frequency is directly proportional to the range of the target, allowing us to estimate the target's distance from the radar with a precision much high than the estimate obtained from the time of flight.



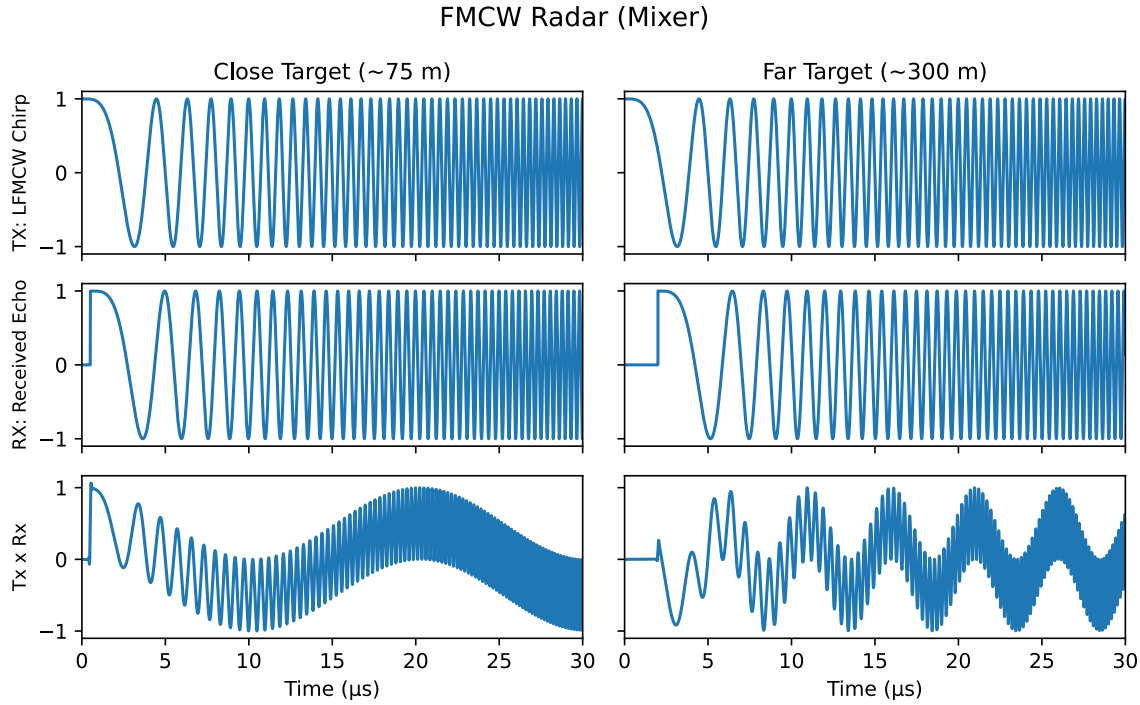


FIGURE 2.1: The transmitted LFMCW chirp (top), delayed echo (middle), and resulting beat signal after mixing and low-pass filtering (bottom), shown for a close target (left, 0.5 µs delay) and far target (right, 2 µs delay). The far target has a higher beat frequency that the close target.

For radar systems where the receiver is close to the transmitter, the transmitter signal can leak into the receiver, creating a strong interference signal that can cover weak pulse echoes from targets. To mitigate this, radar systems rapidly switch between transmission and reception modes so that the transmitter isn't active when receiving.

The compressed pulses are sampled at a lower rate than the original signal, capturing the important beat frequency information.

2.2 Monopulse Radar Hardware

- Radar is a active sensing system that utilises radio waves to determine the distance, direction and velocity of objects in the environment.

- Monopulse radar is a specific radar technique that, allows for the estimation of target position from a single transmitted pulse (as the name suggests). This is achieved by comparing the received signal across multiple receiver antennae simultaneously, rather than sequentially as seen in mechanical scanning radar systems. As a results, monopulse radar can scan targets rapidly and with low latency.
- Another quality of monopulse radar is that it can be implemented using phased array antennas, which allow for electronic beam steering without moving parts.
- While the electronics are complex, the mechanical simplicity of a solid-state radar system can be advantageous for UAV applications, where size, weight and power constraints are critical. This combined with the low latency tracking makes monopulse radar suitable for DAA systems
- This single-look angle measurement capability is valuable for DAA applications, where encounter geometries can change quickly and the sensing system must provide timely and stable bearing information. (TIMING DIAGRAM MAYBE)
- Radar systems are fundamentally limited by the diffraction of radio waves, which sets a lower bound on the angular resolution that can be achieved for a given antenna aperture and operating frequency.
- The angular resolution limit is often expressed as the beamwidth, which is approximately proportional to the wavelength divided by the antenna diameter. This means that for a given frequency, a larger antenna can achieve finer angular resolution. Due to the infeasibility of large antennas on small UAVs, we need to have an angular accuracy that is higher than the beamwidth, which is possible with angle discrimination, a core principle of monopulse radar.
- Figure (MONOPULSE FIG) illustrates how monopulse radar can achieve angle estimation accuracy that is finer than the beamwidth by comparing the received signal across multiple channels. (INCLUDE FIGURE)
- Monopulse processing may be implemented in amplitude-comparison form, phase-comparison form, or hybrid forms depending on the antenna architecture, frequency band, and required accuracy.

- The achievable angular accuracy of a monopulse radar depends on several factors, including antenna aperture, frequency, signal-to-noise ratio, and calibration quality.
- In general, narrower beams and higher signal-to-noise ratios lead to better angle estimation accuracy because the monopulse slope near boresight becomes steeper and measurement noise has less influence.
- Important error sources include thermal noise, receiver imbalance, gain and phase mismatch between channels, quantisation effects, multipath, clutter, sidelobe interference, and target glint.
- Simulation is therefore important for characterising expected performance before hardware deployment.
- The performance of digital processing stages such as amplitude comparison will be evaluated in terms of its resulting angular accuracy, range accuracy, and ability to track multiple targets, under simulation and real world conditions.

2.3 Multi Target Tracking

- Multi-target tracking converts a sequence of noisy radar detections into stable target tracks over time.
- Its purpose is to estimate the current state of each target, including position and velocity, while maintaining continuity between successive radar updates.
- In radar systems, tracking is typically performed after detection and angle estimation, using range, Doppler, and angle measurements as inputs.
- A tracking filter is used to smooth noisy measurements and predict target motion between radar updates.
- Kalman filtering provides a common framework for this process by combining a motion model with uncertain measurements to produce an improved estimate of target state.
- The filter alternates between prediction and update stages, using the predicted target state to guide association and the new measurements to refine the estimate.

$$f(x) = \frac{3}{2}$$

- Variants such as the Extended Kalman Filter may be required when the measurement model is nonlinear.
- Radar measurements are naturally produced in polar coordinates, typically as range, bearing, elevation, and sometimes Doppler.
- For tracking, these measurements may be processed either directly in polar form or transformed into Cartesian coordinates depending on the filter design and motion model.
- The choice of coordinate system affects the linearity of the model, the form of the measurement uncertainty, and the complexity of the update equations.
- Gating is used to restrict which detections are considered plausible matches for an existing track.
- This reduces computational cost and helps reject unlikely associations caused by clutter or nearby targets.
- Data association then determines which measurement should be assigned to each predicted track when multiple candidates are present.

- Track life management controls how tracks are initiated, confirmed, maintained, and deleted over time.
- New detections may first form tentative tracks, which are only promoted to confirmed tracks after consistent observations across multiple updates.
- Similarly, tracks may be deleted after repeated missed detections to avoid maintaining false or stale targets.
- These stages allow the tracker to balance responsiveness to new targets with robustness against clutter and intermittent measurement loss.

2.4 Parallel Processing

- Parallel processing is important in radar systems because many stages of the pipeline, such as FFTs, beamforming, matrix operations, and detection, involve applying the same operation across large volumes of data.
- CPUs, GPUs, ASICs, and other specialised architectures each offer different trade-offs between flexibility, throughput, power efficiency, and development time.
- CPUs are well suited to control logic and sequential tasks, while GPUs are attractive for highly data-parallel workloads and ASICs offer high efficiency at the cost of reduced programmability.
- For UAV radar applications, software-defined GPU-based systems are often preferred because they allow algorithms to be updated more easily than fixed-function hardware.
- GPUs achieve high performance by executing many threads in parallel, making them well suited to radar operations with regular memory access patterns and repeated arithmetic across large arrays.
- Their performance depends not only on raw parallelism, but also on architectural factors such as cache behaviour, memory access patterns, branch divergence, and the choice of warp and block sizes.

- In particular, effective use of L2 cache and predictable control flow can significantly improve throughput, while poor memory locality or divergent branches can reduce performance.
- This architectural understanding helps inform the design of our experiments by motivating attention to data layout, kernel structure, and the balance between computation and data movement on embedded GPU platforms.

CHAPTER 3

Literature Review

3.1 Compact Airborne Radar for UAV DAA

Early demonstrations established that capable radar sensing is achievable under UAV SWaP constraints. Sysak et al. [12] present a lightweight X-band monopulse radar for mid-air collision avoidance on medium UAV platforms, integrating a multi-channel forward-looking antenna with FPGA-based pulse compression on a single 60 gram PCB, with average power below 80 Watts. Field trials demonstrate reliable detection of large obstacles at 1–3 km, and the work validates a simple online pipeline from range compression to TCPA estimation. However, validation is conducted exclusively against large, high radar cross-section targets, and system latency is not reported. This thesis addresses these gaps by characterising detection sensitivity against small moving airborne targets and by measuring end-to-end pipeline latency on embedded hardware.

Wang et al. [7] demonstrate a complete low-SWaP LFMCW system for group 2/3 UAS incorporating FFT-based detection, digital beamforming, multi-target tracking, and GPU-accelerated FFT for update rate improvement. However, performance is validated in simulation only, with no latency measurements, no throughput figures, and no benchmarking of the GPU claim. This thesis addresses these gaps by providing hardware-validated latency and throughput measurements for a comparable pipeline, and by substantiating the GPU acceleration argument with systematic benchmarks.

Moses et al. [13] introduce a 230 gram miniature radar mountable on a small quadcopter, demonstrating that radar hardware can be integrated onto very small platforms. Their evaluation emphasises target identification via micro-Doppler signatures rather than ranging or tracking, leaving open questions about how the small aperture affects range-domain SNR, range resolution, and false alarm behaviour. This thesis addresses this gap by benchmarking range–Doppler processing, CFAR detection, and multi-target tracking under the SNR conditions characteristic of small-aperture UAV radars.

3.2 Embedded Radar Processing Under SWaP Constraints

Shi et al. [14] describe a small FMCW phased array built from off-the-shelf development boards, validating that continuous real-time 2D FFTs are achievable on a 100 MHz FPGA. However, processing was split between an onboard FPGA and offboard post-processing, and signal coherence problems prevented real-world validation. This thesis addresses this gap by implementing a fully onboard, online pipeline and validating it against real target returns.

Newmeyer et al. [15] demonstrated a 120 gram, 8 Watt FMCW phased array pipeline on a Zynq ARM CPU, closely monitoring power, latency distributions, and hardware resources. This work clearly establishes that radar DAA can operate under strict SWaP constraints. However, only one hardware configuration and one algorithm sequence are evaluated. This thesis addresses this gap by systematically comparing alternative algorithmic pathways and evaluating how pipeline bottlenecks change across embedded compute configurations, including GPU-based architectures.

Zhao et al. [16] implement CUDA acceleration for a passive bistatic radar pipeline, reporting up to a 113.13$\times$ speedup over a CPU baseline across three algorithm stages. However, the study covers only clutter suppression, range–Doppler processing, and CA-CFAR detection, omitting angle estimation, coordinate transforms, and Kalman filtering, and does not discuss memory reallocation or thread synchronisation costs. This thesis addresses these gaps by benchmarking a more comprehensive algorithm selection and profiling host–device transfer and synchronisation overhead explicitly.

Zaknoun [17] provides a comprehensive treatment of GPU radar software architecture for real-time processing. However, this work is not situated in a UAV DAA context and does not evaluate performance under UAV SWaP constraints. This thesis adapts these architectural insights to the UAV DAA problem, evaluating how conclusions change under constrained power and memory budgets.

3.3 Sensitivity and Detection in Low-SNR Regimes

A key consequence of SWaP-constrained hardware is reduced aperture, which simultaneously lowers SNR and degrades angular resolution, increasing the likelihood that multiple targets occupy the same resolution cell.

An et al. [18] propose an efficient method for resolving two unresolved targets within a single monopulse cell. However, evaluation is limited to simulated white-noise conditions, the method is not integrated into an end-to-end range–Doppler pipeline, and explicit failure modes exist for same-angle targets and cells containing three or more targets. This thesis addresses these gaps by testing the algorithm under non-white noise conditions and within a complete tracking pipeline, measuring the effect on track quality.

Aubry et al. [19] demonstrate that detection and angle estimation can be treated jointly in a multichannel array, increasing sensitivity to targets not visible in any single channel and avoiding the brittleness of early thresholding under low SNR. However, validation is limited to single simulated targets in Gaussian interference. This thesis addresses this gap by applying the algorithm to multiple targets under non-Gaussian noise regimes and measuring both sensitivity and track-quality impacts alongside compute cost.

3.4 Summary and Research Gap

Across the UAV DAA radar literature three gaps emerge. First, existing hardware demonstrations validate detection against large, high radar cross-section targets only; no work characterises sensitivity to small moving airborne targets under realistic encounter conditions.

Second, processing feasibility on constrained embedded hardware has been demonstrated on FPGAs and ARM CPUs, but no work benchmarks a GPU-based pipeline in a UAV DAA context with comprehensive algorithm coverage and architectural profiling. Third, sensitivity algorithms for low-SNR monopulse radar exist but have only been validated under idealised white Gaussian noise with single targets, with no integration into complete pipelines or evaluation under realistic multi-target conditions.

This thesis addresses all three gaps by characterising detection and tracking sensitivity against small moving airborne targets, by benchmarking a comprehensive selection of radar processing and tracking algorithms on embedded GPU hardware with explicit profiling of host–device transfer and synchronisation overhead, and by evaluating monopulse sensitivity algorithms within a complete multi-target tracking pipeline under realistic noise conditions.

CHAPTER 4

Methodology

4.1 Overview

- The methodology of this thesis involves designing and implementing parts of a software-defined radar processing pipeline for a small monopulse radar system, and evaluating its performance in terms of both computational efficiency and tracking accuracy.
- The first sections in this define the pipeline, its applications and their constituent operations. The questions and solution space for each operation is outlined, and the questions mirror the gaps identified in the literature review.
- Then the section describes the datasets that will be used for testing, which include both synthetic data and real radar data recorded from our AWR2243 radar system.
- Lastly the section outlines the difference techniques used to evaluate the performance of the software pipeline, the metrics we will report, and the tools we will use for performance analysis.
- The evaluation procedure for each operation in our pipeline is outlined in Sections 4.3, and 4.4.
- The experiments are designed to fill the identified gaps in the literature.

4.2 Software Design

The software processing pipeline is designed to receive the radar data from four Texas Instruments AWR2243 radar chips, and produce tracks of targets in the radar's field of view.

The software is designed to be modular, with each stage of the pipeline performing a specific function. The pipeline is comprised of three applications, a pre-processing applications which is concerned with preparing the raw radar data for detection, an aggregator application, which is concerned with combining the processed data from the four video streams into a single stream, and a detection and tracking application, which is concerned with detecting targets in the radar data and tracking them over time. This application level design is depicted in Figure 4.1.

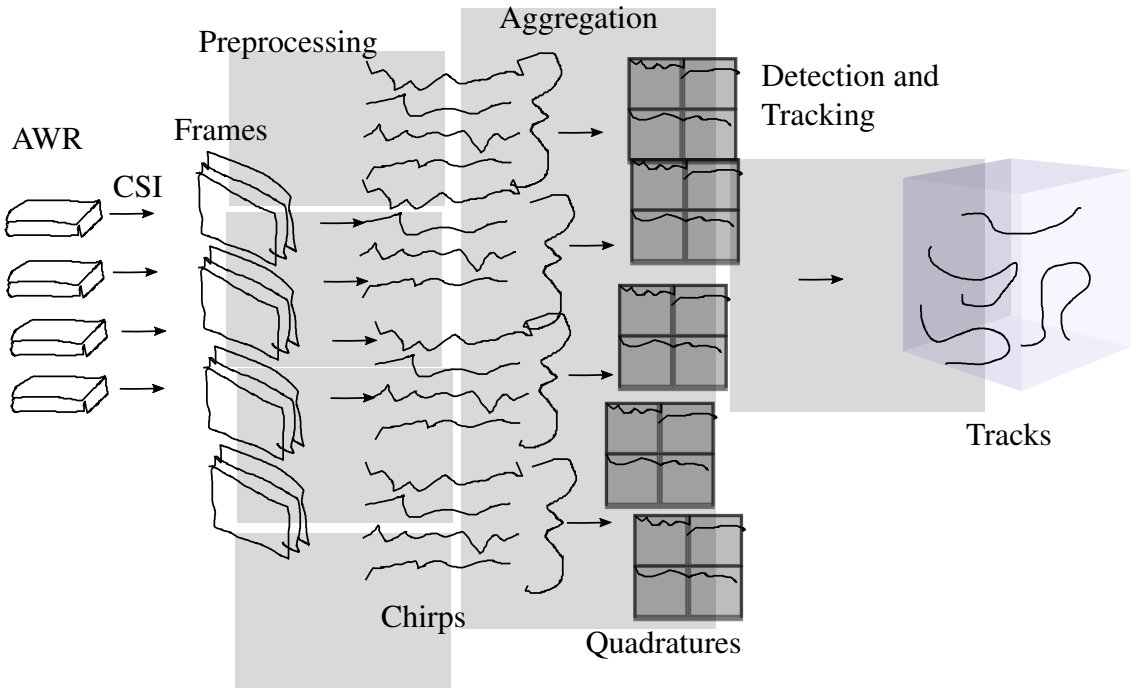


FIGURE 4.1: Application level dataflow. The pre-processing application receives the AWR2243's raw radar data over four CSI video data streams. The data is organised into frames. The processing application then outputs the data in the form of chirps containing target ranges. The aggregator application receives the four processed data streams, and combines them into a single data stream organised by quadruplet, a collection of four receivers necessary for monopulse angle sensing. The detection and tracking application receives a stream of quadruplets, and produces tracks of targets in the radar's field of view.

4.2.1 Data Types and Protocols

The specifications for the data types and protocols used for communication between the different stages of the pipeline are outlined in the Appendix.

TODO: Add them to Appendix, what is a chirp?, what is a quadruplet?, how are the organised in memory?, how are they transmitted between stages?

4.2.2 Preprocessing Application

The preprocessing application is responsible for receiving the raw radar data from the AWR2243 radar chips, and preparing it for detection. An instance of the application runs per incoming video stream. The application performs the following operations sequentially: formatting, windowing, FFT, and coherent integration, see Figure 4.2. The implementation and experiment details for these operations are outlined in Section 4.3.

4.2.3 Aggregator Application

4.2.4 Detection and Tracking Application

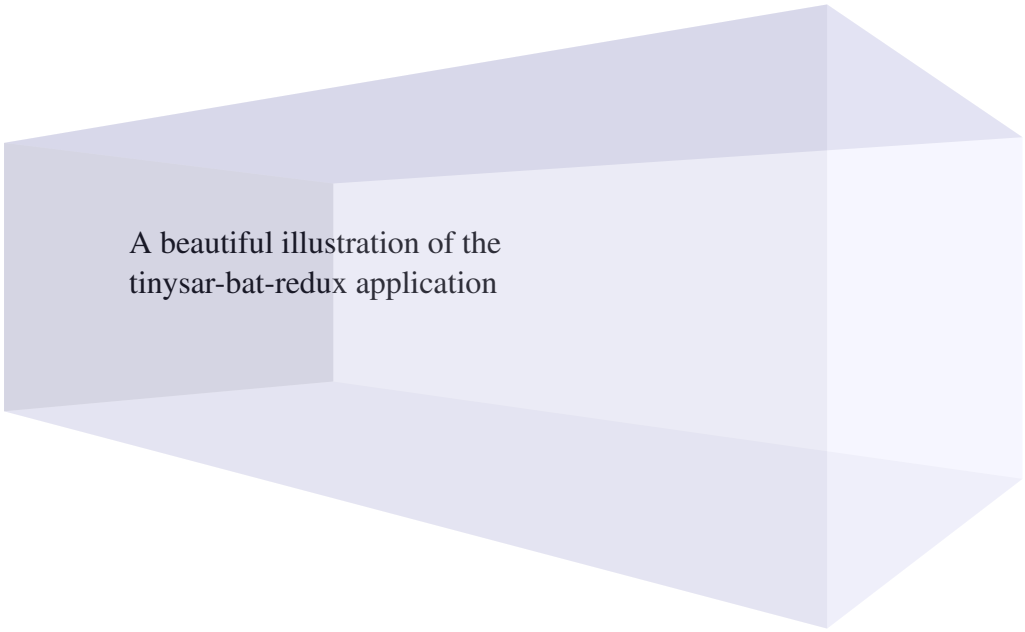


FIGURE 4.2: Dataflow for the preprocessing application. Multiple instances of the application run on separate processes, with each instance receiving compressed baseband radar data samples from an incoming video stream. Formatting organises samples into contiguous blocks in memory, and discards empty chirps. The windowing stage applies a window function to the data to reduce spectral smearing in the next state. The FFT stage applies a Fourier Transform across fast time into the frequency domain, revealing target range. The coherent integration stage sums the complex FFT values of identical pulses within the same frame.

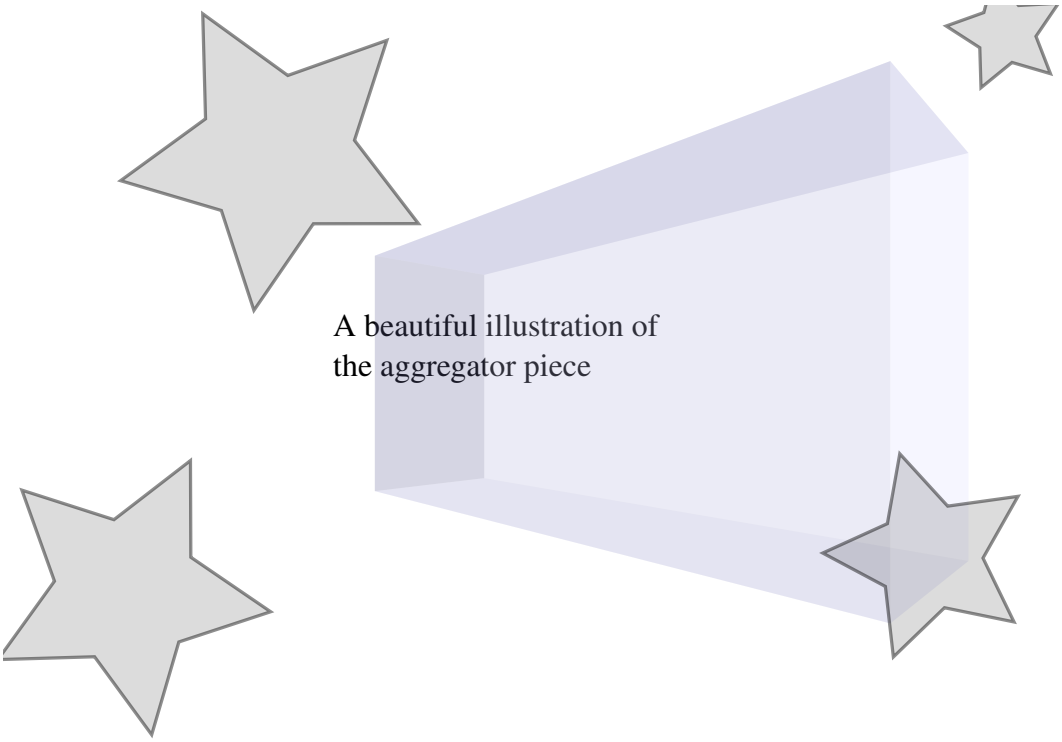


FIGURE 4.3: Dataflow for the aggregator application. The application receives the four processed data streams from the preprocessing application, and combines them into a single data stream organised by quadruplet, a collection of four receivers necessary for monopulse angle sensing.

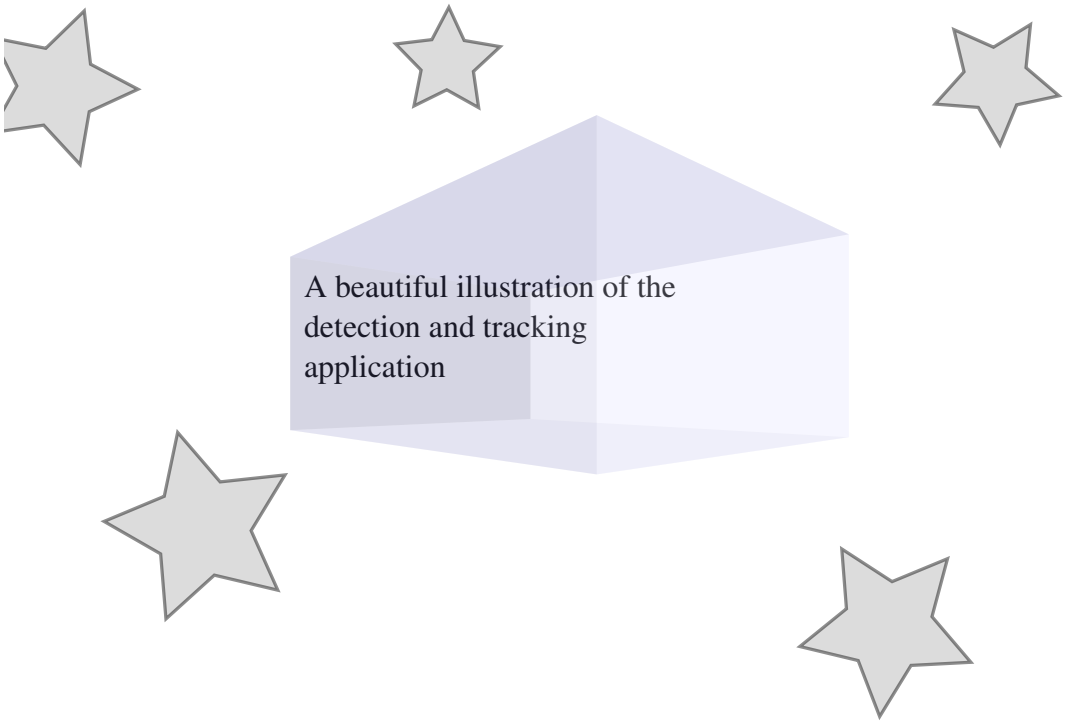


FIGURE 4.4: Dataflow for the detection and tracking application. The application receives a stream of quadruplets, and produces tracks of targets in the radar's field of view. By applying the following operations: CFAR, monopulse angle sensing, polar transform, data association, and tracking filters.

4.3 Processing Application Evaluation

4.3.1 Windowing

To reveal target range from a single radar pulse, an FFT (Fast Fourier Transform) is applied to the dechirped pulse. When an FFT is performed on a finite signal, the theoretically infinite signal is implicitly multiplied by a rectangular window. A term-wise multiplication in time corresponds to a convolution in the frequency domain. Since a rectangular window has large spectral sidelobes, it causes smearing of the signal's true frequency content. By applying a more suitable window function prior to the FFT, this spectral smearing can be reduced, as shown in Figure 4.5.

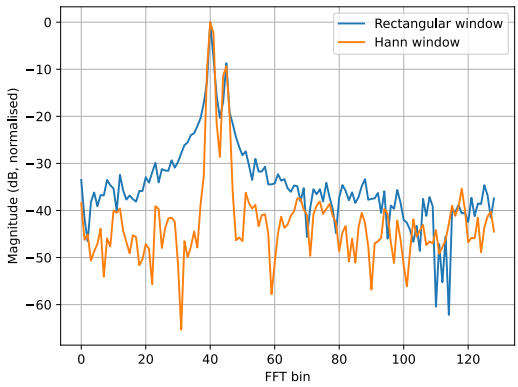


FIGURE 4.5: Results of an FFT performed on a signal with two frequencies and Gaussian noise, with and without a Hann window. Without the Hann window, the bins close to the peak frequency bins 40 and 45 are significantly larger than bins further away, misrepresenting the homogeneity of the Gaussian noise.

This thesis evaluates a range of practical window functions and their implementation on CPU and GPU. Candidate windows include common real-time tapers such as Hann and Hamming, lower-sidelobe windows such as Blackman and Blackman–Harris, and adjustable-parameter windows such as Kaiser, which allow explicit control over sidelobe level and mainlobe width. Because windowing is a point-wise multiplication, it is naturally suited to single instruction, multiple data (SIMD) execution.

Performance profiling measures the latency contribution and throughput associated with each window and implementation method. A cost-benefit analysis weighs the latency cost against downstream benefits of a cleaner signal, including reduced sidelobe-induced false positives. These procedures provide a basis for selecting windows that are spectrally well behaved, computationally efficient, reduce detection false alarms, and reduce the computational workload of downstream algorithms such as CFAR.

4.3.2 Coherent Integration

Because aerial hazards are often small and distant, the SNR of their radar returns can be indistinguishable from noise. After the FFT, the signal represents the intensity of reflection as a function of range, where the phase of each sample corresponds to the phase of the received signal, a value necessary for phased-based angle discrimination. By summing the complex range bin values of identical pulses close together in time, the effective signal to noise ratio is improved while the phase information is preserved. Careful attention towards not summing different types of pulses together is required. The process of coherent integration is depicted in Figure 4.6.

Different integration window sizes are compared on real data, and the resulting changes in detection sensitivity and false alarm behaviour are measured. CPU and GPU implementations are also compared, and the downstream effect on angle sensing stability is quantified. An ablation study holds the window, FFT size, and CFAR parameters fixed while varying only the integration policy, isolating its effect. The contribution of the integration stage to the overall latency and throughput of the pipeline is also measured.

4.4 Tracking Application Evaluation

4.4.1 CFAR

Constant False Alarm Rate (CFAR) detection is a key mechanism for resolving a map of noisy range data into discrete detections given a target false alarm rate. It does this by masking

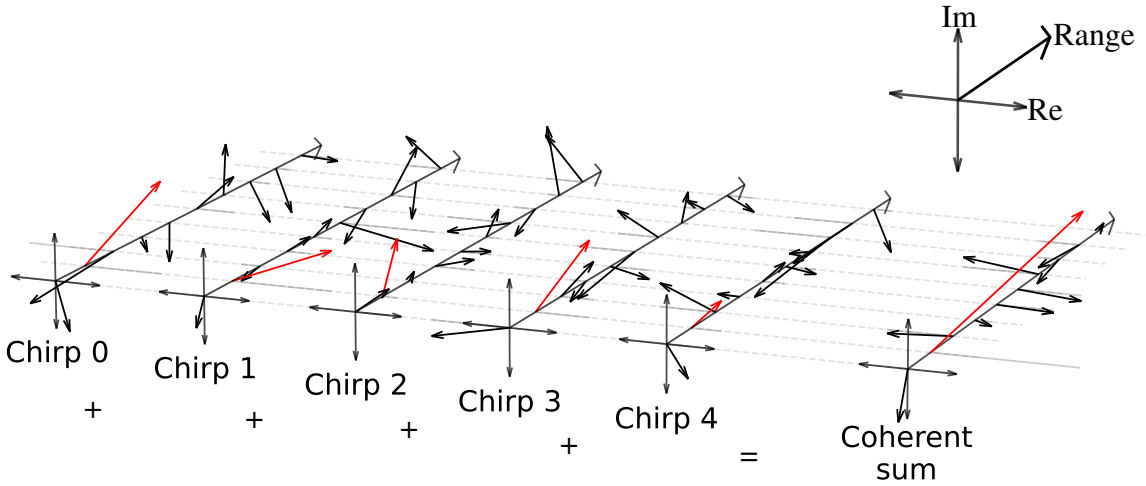


FIGURE 4.6: Coherent integration across five chirps for each range bin. Each arrow represents the complex FFT value of a single bin, with the real and imaginary components shown on the horizontal and vertical axes of each chirp panel. Most bins contain noise-like phasors with varying direction, so their coherent sum remains small. The highlighted bin contains a weak but consistent signal component across chirps, so its phasors add constructively, producing a larger resultant in the coherent-sum panel.

out any samples with magnitude below a threshold determined by the power of the samples around it (see Figure 4.7). Threshold is calculated as,

$$T = \alpha \cdot \frac{1}{N} \sum_{i=1}^{N} x_i \tag{4.1}$$

Where T is the threshold, α is a factor determined by the desired false alarm rate, N is the number of training samples, and x_i are the power (magnitude squared) of the training samples. Under a Gaussian noise assumption, the coefficient α that leads to a desired false alarm rate P_{fa} can be calculated as,

$$P_{fa}(\alpha; N) = \left(\frac{N}{N+\alpha}\right)^N \sum_{k=0}^{3} \binom{N+k-1}{k} \left(\frac{\alpha}{N+\alpha}\right)^k \tag{4.2}$$

where the summation to $k = 3$ arises because the cell under test is formed by summing the power from four receiver channels, giving a chi-squared distribution with 8 degrees of

freedom. Unlike the single-channel case, α cannot be rearranged into a simple closed-form expression, so it is computed numerically.

For a desired false alarm rate $P_{fa}^{\star}$, α is found by solving

$$P_{fa}(\alpha; N) = P_{fa}^{\star} \tag{4.3}$$

using bisection. The method begins with $\alpha_{lo} = 0$ and increases an upper bound α_{hi} until $P_{fa}(\alpha_{hi}; N) \leq P_{fa}^{\star}$. It then repeatedly bisects the interval and evaluates $P_{fa}(\alpha; N)$ at the midpoint, updating the lower or upper bound depending on whether the resulting false alarm rate is above or below the target. This continues until the interval is sufficiently small. In this way, α is obtained as the numerical solution that gives the required false alarm rate.

The correctness of the implementation is verified by comparing the desired false alarm rate to the actual false alarm rate when there is no target signal present when alpha is calculated under the assumption of a chi-squared distribution.

Three different implementations of CFAR are compared for their latency, a naive implementation on the CPU, an implementation that uses a prefix sum table, and a naive solution on the GPU. Additionally different noise assumptions are tested. The naive implementation involves calculating the noise statistic with a repeated sum per sample of interest. The prefix sum table method reduces the number of sums required to four per sample of interest regardless of the number of training cells (see Figure 4.4.1). The GPU implementation creates a thread per sample of interest (see Figure 4.4.1). A sweep of window size, frame buffer size and false alarm rate values is conducted for each CFAR implementation on real and simulated data. The performance (throughput, latency, memory, false alarm rate, sensitivity) is compared across configurations in a cost benefit analysis. This addresses the literature gap in understanding how detection sensitivity and false alarm behaviour trade off against latency and throughput under SWaP constraints.

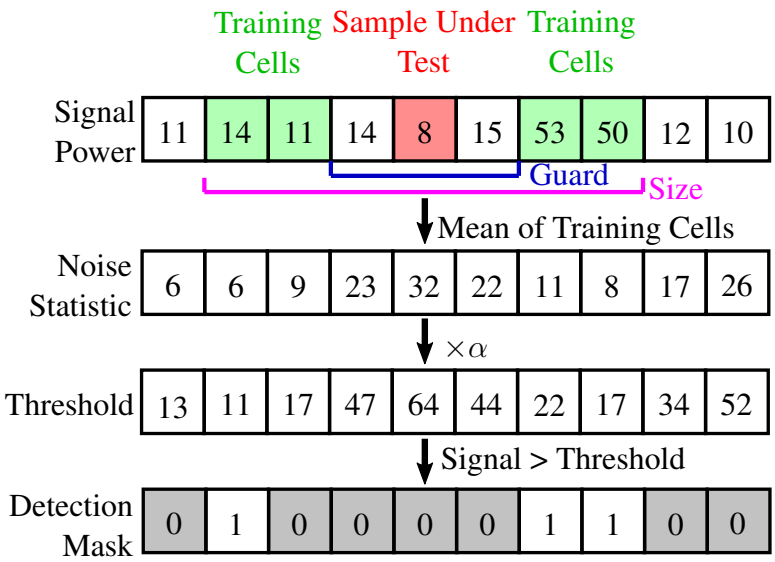


FIGURE 4.7: Naive Implementation of CFAR for the CPU. Each sample is compared to a threshold determined by the power of the samples around it. The threshold is calculated by taking the average power of the surrounding samples and multiplying it by a factor α determined by the desired false alarm rate.

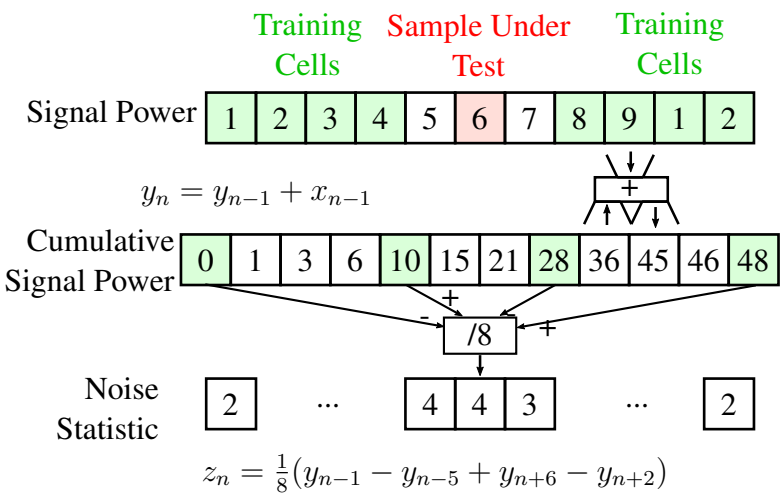


FIGURE 4.8: CFAR implementation using a prefix sum table (cumulative signal power). The prefix sum table allows the noise statistic to be calculated with a fixed number of sums per sample of interest, regardless of the number of training cells. In this specific case, after the cumulative signal power is calculated in preprocessing, it only takes four sums of to find the sum of eight training cells.

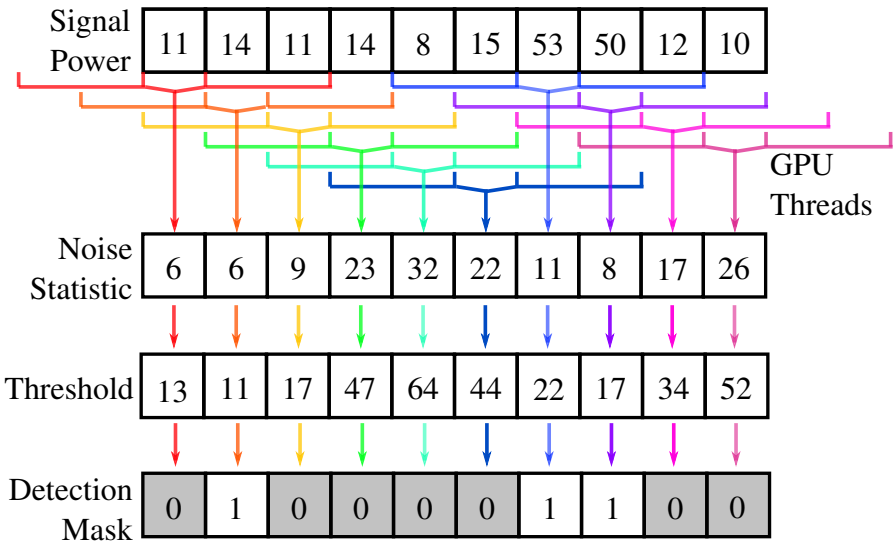


FIGURE 4.9: CFAR implementation on the GPU. Threads (each depicted as a unique colours) are each responsible for a sample, allowing for parallel processing of the CFAR algorithm across the entire range profile. Each thread calculates its own noise statistic, threshold, and mask.

4.4.2 Monopulse Angle Sensing

Angle sensing converts multi-channel measurements into angle estimates that drive association and tracking. We will assess candidate angle discriminators by running controlled sweeps over SNR, channel imbalance, and multi-target proximity, and measuring angular bias, mean squared error, and confidence. The latency and throughput of CPU and GPU implementations will be measured. We will explicitly quantify the time and compute costs of stabilisation strategies such as sum-channel gating, regularised ratios, and calibration placement. The "Amplitude Angle Discriminator" and "Phase Angle Discriminator" methods will be compared for their computation requirements, angular bias and variance [20]. The algorithm will be validated on data obtained from flight tests. This addresses the gap in validating monopulse angle estimation methods beyond idealised simulations, which often assume Gaussian noise and clean single detections per target.

4.5 Testing Datasets

The datasets described below are used for the experiments outlined in the following sections (see Sections 4.3, and 4.4)

- **Generic Sine Waves with Noise**: To validate the correctness of our processing algorithms as well as their speed, we generate binary files of synthetic radar data consisting of a complex sine wave with additive white Gaussian noise. This allows us to test the software under controlled conditions where we can precompute the expected output, and to evaluate the performance of the software in terms of latency and throughput without the confounding factors of real-world data. (INCLUDE FIGURE HERE illustrating the signal)
- **AWR2243 Recorded Data**: To validate the operation of our pipeline with realistic data speeds, timing uncertainties, and electrical noise, we record data from our AWR2243 radar system in TEST_SOURCE mode. In test source mode, two targets can be simulated on the chip with position velocity and bounds. The receivers record what the reflected signals from these targets would look like. While the data recorded is not fully representative of real-world data, it allows us to test the software with realistic data rates and noise characteristics.

4.6 Performance Analysis

4.6.1 Throughput Analysis

TODO: Measure in Bytes Per Second

4.6.2 Latency Analysis

TODO: Use NSight

TODO: Ablation Study

TODO: Explore the effect of different hardware contexts on the performance. What contexts prevent us from implementing certain solutions and why

4.7 Computational Performance Metrics and Analysis Methods

- This section evaluates the computational performance of the radar pipeline, focusing on throughput and latency.
- The aim is to determine whether the implemented algorithms are suitable for real-time UAV DAA applications.
- Throughput analysis measures how much data each implementation can process per unit time, reported in bytes per second.
- This shows whether the pipeline can sustain the required radar data rate.
- Latency analysis measures the time taken for data to pass through each stage of the pipeline and through the system as a whole.
- NVIDIA Nsight is used to indentify bottlenecks, memory transfer costs and kernel execution overheads.
- Variations of each operation are compared to understand the impact of different algorithmic choices and optimisations on performance.
- An ablation study is used to evaluate the impact of individual optimisation choices by noting how the performance of the pipelines change when specific operations are removed from the process.

- This helps identify which design decisions provide meaningful performance gains, and where further optimisation efforts should be focused. A table will illustrate the latency and throughput contribution of each stage of the pipeline.
- Performance is also compared across different hardware contexts to understand how architectural constraints affect implementation choices and runtime behaviour, e.g. physically unified memory vs managed memory.
- The Xavier has physically unified memory and the desktop environment has managed memory, with a Ryzen "Threadripper" 3960X CPU and an NVIDIA RTX A6000 GPU.
- This informs which solutions are practical on embedded and GPU-based radar platforms.

4.8 Tracking Performance Metrics and Analysis Methods

- This section evaluates how different algorithmic choices affect tracking performance, not just computational speed.
- In addition to runtime analysis, we compare the effect of each design on sensitivity in detection and tracking tasks.
- An ablation study is used to test alternative implementations for key operations and measure their impact on tracking quality.
- This allows us to identify which optimisations preserve accuracy and which introduce unacceptable performance loss.
- After both computational and sensitivity analysis, a trade-off table is produced to compare candidate pipeline designs.
- This table summarises the computational cost and tracking performance of each design and is used to inform the final embedded implementation.

4.9 Results

4.10 Processing Application

4.10.1 Windowing

No results yet

4.10.2 Coherent Integration

- Show that the coherent integration implementation is correct and working on both dataset, show the different in SNR visually with waterfall plots.
- Show the Nsight results for the different implementations, and show the latency contribution of the integration stage to the overall pipeline.
- Show the effect of different integration window sizes on detection sensitivity and false alarm rate.

4.11 Tracking Application

4.11.1 Constant False Alarm Rate

4.11.1.1 Correctness

Under the assumption of an 8-degrees of freedom chi-squared distribution of quadruplet sample noise power, the CFAR threshold coefficient α is calculated numerically for a range of false alarm rates. The actual false alarm rate is then measured by running the CFAR algorithm on the synthetic dataset with an SNR of zero. Figure 4.10 illustrates the results.

The results indicate that the numerical solution for α is accurate, with the actual false alarm rate closely matching the desired. The successful performance of CFAR with a false alarm rate of 0.001 is illustrated in Figure 4.11.1.1.

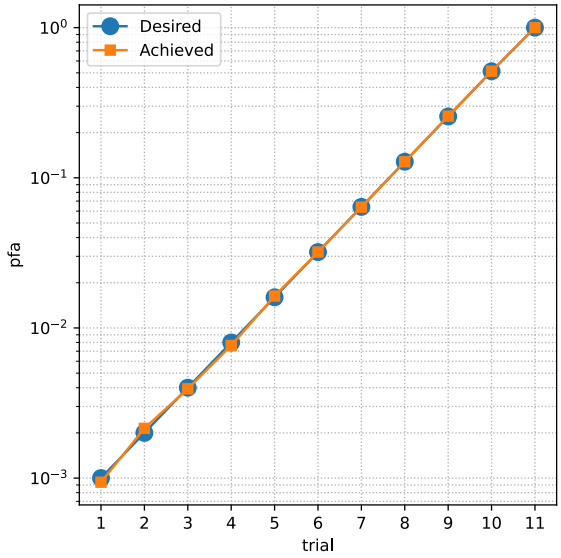


FIGURE 4.10: Results

4.11.1.2 Computational Performance

The speed of each of the three CFAR implementations is measured across a range of buffer sizes, from one frame of 767 samples, to 4042 frames of 767 samples. This sweep is performed on both the desktop and the Xavier (as described in Section 4.6.2). The results are illustrated in Figures 4.13 and 4.12.

The speed is also plotted as a function of the number of CFAR training cells. The results are illustrated in Figures 4.15 and 4.14. The results illustrate that both the GPU and the prefix sum table implementations are not significantly affected by the number of training cells, while the naive CPU implementation's speed increases linearly with the number of training cells.

TODO: Run window size experiment on Xavier.

- Show results for memory usage
- Show results for NVIDIA NSight
- Show ablation study results

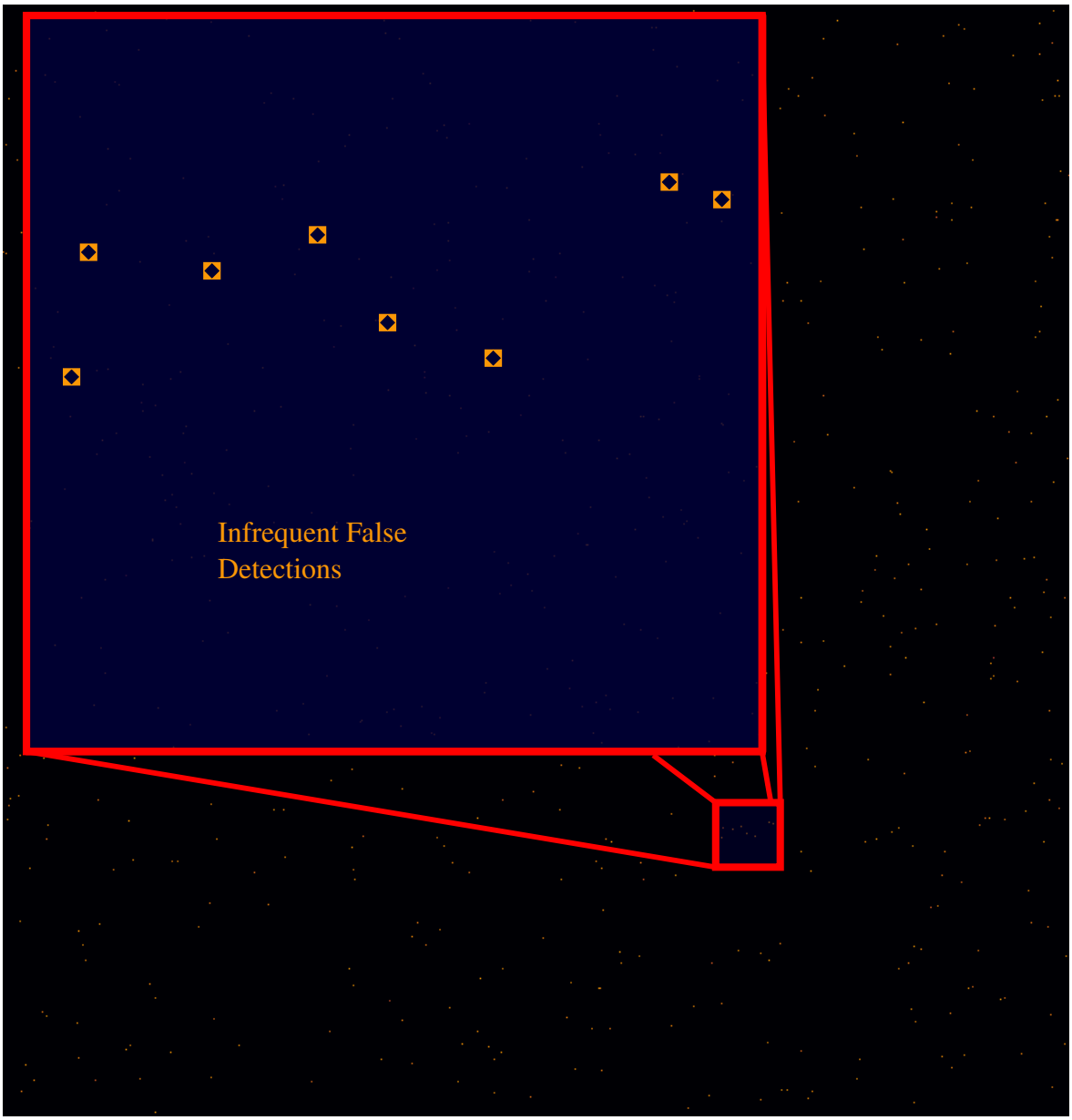


FIGURE 4.11: CFAR with a desired false alarm rate of 0.001 infrequently detect peaks within a target less noise as expected. The figure is a waterfall plot, with chirp on the y-axis and range on the x-axis. The brightness of the cell reflects the power of the corresponding chirp's range bin. The black space indicates that most cells are masked by CFAR.

4.11.2 Monopulse Angle Sensing

- Show results for monopulse, correctness, latency for different implementations, and ablation study results.

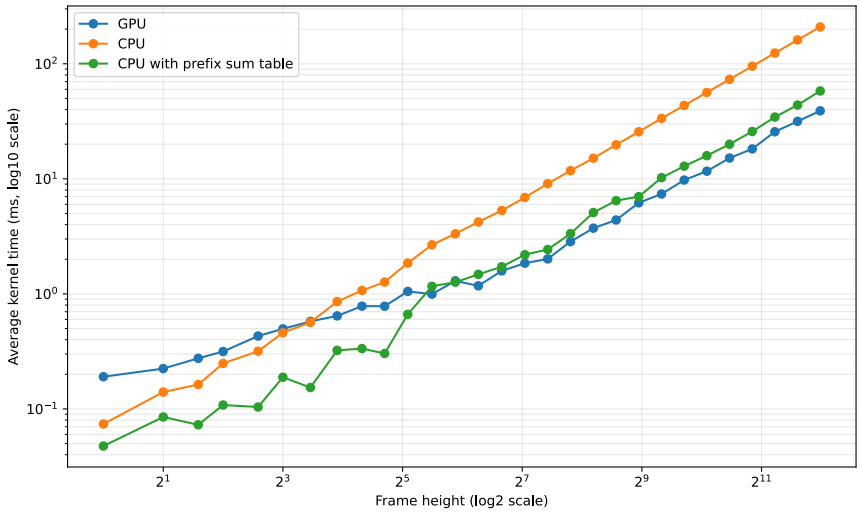


FIGURE 4.12: Computational Performance of three CFAR implementations as a function of buffer size on the desktop setup. The CPU implementation with a prefix sum tables is the fastest for small buffer sizes, but the GPU implementation is slightly faster for larger buffer sizes. The naive CPU implementation is the slowest across all buffer sizes.

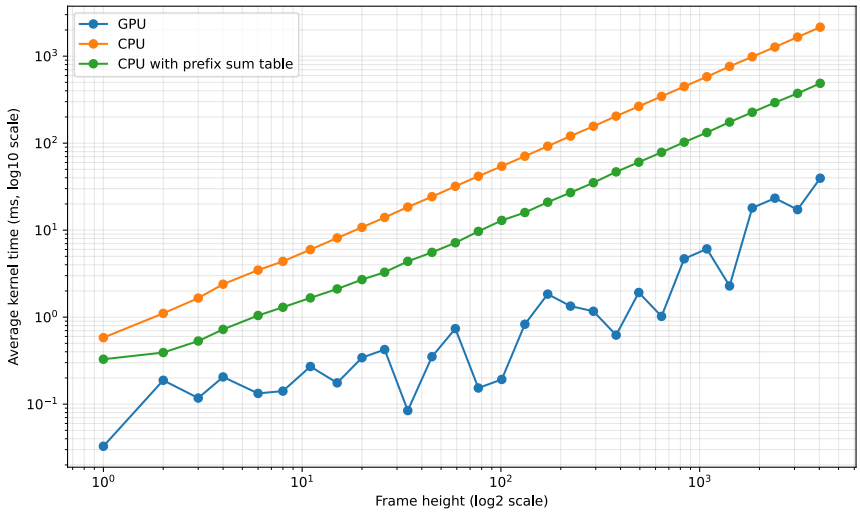


FIGURE 4.13: Computational Performance of three CFAR implementations as a function of buffer size on the Xavier setup. The CPU implementation with a prefix sum tables is the fastest for small buffer sizes, but the GPU implementation is slightly faster for larger buffer sizes. The naive CPU implementation is the slowest across all buffer sizes.

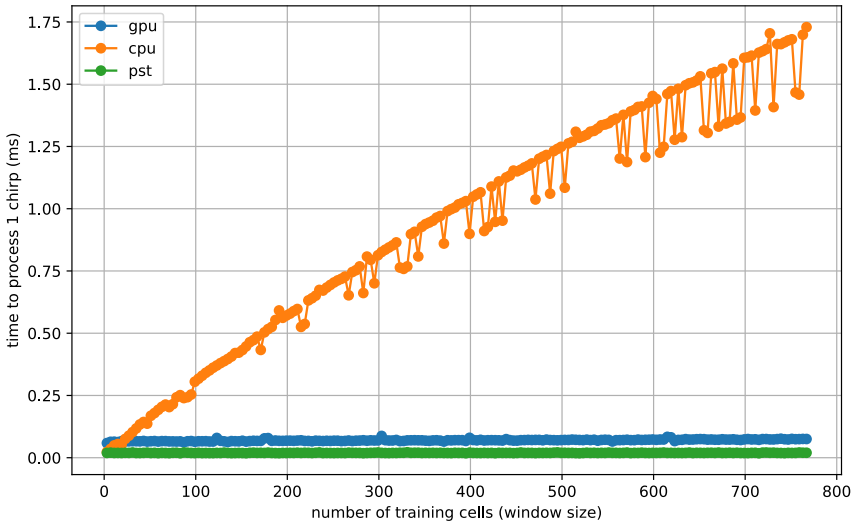


FIGURE 4.14: Computational Performance of three CFAR implementations as a function of the number of training cells on the Desktop Setup. Buffer size is fixed at 10 quadruplets.

FIGURE 4.15: Computational Performance of three CFAR implementations as a function of the number of training cells on the Xavier Setup. Buffer size is fixed at 10 quadruplets.

- Memory usage and NSight results for different implementations.

CHAPTER 5

Discussion

5.1 Processing Application

TODO: Discuss the results for each of the operations. relate to our contributions, gaps and results from the other areas of research.

5.1.1 Windowing Operation

- Discuss the effects that the different windowing functions have the reliability of CFAR on the synthetic dataset, and the real-world dataset.
- Discuss the computational performance of the different windowing functions, and how this affects the overall performance of the pipeline.
- Discuss the implications of these results for the design of radar processing pipelines, and how they inform our understanding of the trade-offs between different windowing functions in terms of detection performance and computational efficiency.

5.1.2 Coherent Integration Operation

- Discuss the effects that the different coherent integration window sizes have on the SNR of the output, and how this affects the sensitivity of the target to CFAR detection on the synthetic dataset.
- Discuss the computational performance of the different coherent integration window sizes, and how this affects the overall performance of the pipeline.

- Discuss the implications of these results for the design of radar processing pipelines, and how they inform our understanding of the trade-offs between different coherent integration window sizes in terms of time resolution, computational efficiency, and detection performance.

5.2 Tracking Application

TODO: Discuss the results for each of those. relate to our contributions, gaps and results from the other areas of research.

5.2.1 CFAR Operation

- Discuss the correctness of the CFAR implementation, and how the results from the synthetic dataset validate our implementation.
- Discuss the computational performance of the three CFAR implementations, and how they scale with buffer size and number of training cells. Discuss how the GPU implementation is faster for larger buffer sizes, but the prefix sum table implementation is faster for smaller buffer sizes. Discuss how the naive CPU implementation is significantly slower than the other two implementations, and how its performance degrades with increasing buffer size and number of training cells.
- Discuss the implications of these results for the design of radar processing pipelines.

5.2.2 Monopulse Angle Sensing Operation

- Our results indicate that the monopulse angle sensing implementation is correct, and provides accurate estimates of targets within the require range and signal to noise ratio. Although only one implementation was explored, the strong results provide a baseline for future work to explore different implementations and optimisations.

5.3 Addressing the Identified Knowledge Gaps

Although our work has made progress in building a radar processing pipeline suitable for small UAVs, the main knowledge gaps identified in Chapter 2 still remain. In this section, we will discuss how our work has addressed these gaps, and what further research is needed to fully address them.

- Our work has made progress towards end-to-end systems level comparisons under SWaP constraints by implementing and evaluating a complete radar processing pipeline on embedded GPU hardware. However, further research is needed to explore a wider range of algorithm combinations and optimisations, and to evaluate the performance of the pipeline in more realistic encounter scenarios with small targets.
- Our experimental design has included small moving aerial targets and encounter-like scenarios in both simulation and real data collection, which has provided a more realistic evaluation of the performance of our radar processing pipeline. However, further research is needed to explore a wider range of target types, encounter scenarios, and environmental conditions to fully understand the performance of the pipeline in real-world settings.
- Our work has provided GPU-oriented onboard pipeline evidence by implementing and evaluating our radar processing pipeline on embedded GPU hardware, and by quantifying the impacts of data movement costs and architectural constraints on performance. However, further research is needed to explore a wider range of hardware configurations, power modes, and clock settings to fully understand the implications of these factors for real-time UAV DAA applications.
- Our experiments have been repeated across multiple hardware contexts, including CPU-only, unified-memory embedded GPU, and discrete CPU+GPU settings where available. This has provided insights into how architectural constraints affect implementation choices and runtime behaviour. However, further research is needed

to explore a wider range of hardware configurations and to evaluate the performance of the pipeline under different power and clock modes to fully understand the implications for embedded UAV radar platforms.

CHAPTER 6

Conclusion

6.1 Summary of Findings

TODO: Summarise the finds at a high level. State the best/recommended configuration.

TODO: Link together all the sections of the discussion into main takaways. Kinda like an executive summary.

A modular radar pipeline for small-UAV detect-and-avoid was successfully designed and evaluated.

CFAR was validated, with achieved false alarm rates closely matching desired values.

Implementation choice strongly affected runtime, latency, and throughput across hardware platforms.

Hardware context changed which implementation was most suitable, supporting hardware-aware pipeline design.

6.2 Industry Partner Outcomes

TODO: Without sharing anything sensitive, detail how the research done benefits mission systems in their project goals.

TODO: Document tests and downstream work that my work enabled.

TODO: Discuss how this technology will after the industry outside of Mission Systems too.

Delivered practical benchmarking evidence to guide embedded radar software design choices.

Helped identify implementation trade-offs relevant to Mission Systems' UAV radar pipeline.

Produced a reusable experimental and profiling framework for future internal development.

Contributed design guidance with relevance beyond the immediate industry project.

6.3 Future Work

TODO: What about our research was successful, what wasn't successful

TODO: Discuss the limitations of the set of computing hardware that we were able to explore

TODO: We only explored one radar device

TODO: Talk about algorithms that were not explored

TODO: Compare to other sensing systems

TODO: Sensor Fusion

(See Addressing Knowledge Gaps in Chapter 6.)

Bibliography

[1] European Union Aviation Safety Agency (EASA). (Jul. 2024). Easy access rules for unmanned aircraft systems (regulations (eu) 2019/947 and 2019/945): Revision from july 2024. Online publication page (Revision from July 2024), (visited on 04/02/2026).

[2] RTCA, Inc. (Dec. 2025). Sc-228: Minimum operational performance standards for unmanned aircraft systems. Committee page; Terms of Reference listed as December 16, 2025, (visited on 04/02/2026).

[3] C. Serres, B. Gill, M. W. M. Edwards and M. G. Wu, 'Detect-and-avoid safety and operational suitability analysis using an electro-optical/infrared sensor model,' National Aeronautics and Space Administration, NASA Technical Memorandum NASA/TM–20210016872, Mar. 2021, Ames Research Center; MIT Lincoln Laboratory.

[4] B. Borden, *Radar Imaging of Airborne Targets: A Primer for Applied Mathematicians and Physicists*. CRC Press, 1999, Originally published 1999; later reprints available via CRC Press/Taylor & Francis.

[5] W. A. Lies, L. Narula, P. A. Iannucci and T. E. Humphreys, 'Low SWaP-C radar for urban air mobility,' in *Proceedings of the 2020 IEEE/ION Position, Location, and Navigation Symposium (PLANS)*, Preprint available at https://rnl.ae.utexas.edu/images/stories/files/papers/lies2020lowSwapc.pdf, Portland, OR, Apr. 2020.

[6] T. McLain, L. R. Sahawneh, M. O. Duffield and R. W. Beard, 'Detect and avoid for small unmanned aircraft systems using ADS-B,' *Air Traffic Control Quarterly*, vol. 23, no. 2–3, pp. 203–224, 2015, BYU ScholarsArchive preprint: https://scholarsarchive.byu.edu/facpub/1895.

[7] Z. Wang, X. Lin, X. Xiang, E. Blasch, K. Pham, G. Chen, D. Shen, B. Jia and G. Wang, 'An airborne low SWaP-C UAS sense and avoid system,' in *Proceedings of SPIE,*

Sensors and Systems for Space Applications IX, vol. 9838, Baltimore, MD, May 2016, p. 98380C.

[8] Echodyne Corp., *EchoFlight 4d airborne radar: Product datasheet*, Specifications: K-band 24.45–24.65 GHz; weight 817 g; 45 W operating power; size 18.7 cm × 12 cm × 4 cm. Retrieved 25 March 2026 from https://www.echodyne.com/media/ec0jv5tp/ts-echoflight-echodyne.pdf, 2024.

[9] Mordor Intelligence. (Jul. 2025). Airborne radars market size & analysis - growth trends & forecasts (2025 - 2030). Page last updated on July 24, 2025, (visited on 04/02/2026).

[10] K. Zaknoun, 'Real-time, gpu-accelerated processing of digital radar signal data,' Master's thesis, University of Turku, Jun. 2024.

[11] W. Feng, X. Hu and X. He, 'Artificial intelligence (ai)-based radar signal processing and radar imaging,' *Electronics*, vol. 13, no. 21, 2024.

[12] D. Sysak, A. Byndas, T. Karas and G. Jaromi, 'Lightweight and compact pulse radar for uav platforms for mid-air collision avoidance,' *Sensors (Basel)*, vol. 25, no. 23, p. 7392, Dec. 2025.

[13] A. Moses, M. J. Rutherford and K. P. Valavanis, 'Radar-based detection and identification for miniature air vehicles,' in *2011 IEEE International Conference on Control Applications (CCA)*, 2011, pp. 933–940.

[14] L. Shi, C. Allen, M. Ewing, S. Keshmiri, M. Zakharov, F. Florencio, N. Niakan and R. Knight, 'Multichannel sense-and-avoid radar for small uavs,' in *2013 IEEE/AIAA 32nd Digital Avionics Systems Conference (DASC)*, 2013, 6E2-1-6E2–10.

[15] L. Newmeyer, D. Wilde, B. Nelson and M. Wirthlin, 'Efficient processing of phased array radar in sense and avoid application using heterogeneous computing,' in *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*, 2016, pp. 1–8.

[16] X. Zhao, P. Liu, B. Wang and Y. Jin, 'Gpu-accelerated signal processing for passive bistatic radar,' *Remote Sensing*, vol. 15, no. 22, 2023.

[17] K. Zaknoun, 'Real-time, gpu-accelerated processing of digital radar signal data,' Master of Science in Technology Thesis, University of Turku, Turku, Finland, 2024.

[18] Q. An, C. Yeh, Y. Lu, Y. He and J. Yang, ‘Time-varying angle estimation of multiple unresolved extended targets via monopulse radar,’ *IEEE Transactions on Aerospace and Electronic Systems*, vol. 60, no. 4, pp. 4650–4667, 2024.

[19] A. Aubry, A. D. Maio, S. Marano and M. Rosamilia, ‘Single-pulse simultaneous target detection and angle estimation in a multichannel phased array radar,’ *IEEE Transactions on Signal Processing*, vol. 68, pp. 6649–6664, 2020.

[20] MIT Lincoln Laboratory, *Tracking and parameter estimation*, Lecture 9 slides, *Introduction to Radar Systems*, Marked "NBP 7/31/2008" on the title slide; slide identifier EST&TRACK-1, Jul. 2008.

APPENDIX A

Appendix

A1 Case Studies

A2 Case Studies: Learning Objectives Covered by Thesis

For ESIPS substitution, this thesis is proposed to replace content from two elective units: **AMME5520 Advanced Control and Optimisation** and **ELEC3305 Digital Signal Processing**. The proposed thesis covers over 75% of learning objectives from each unit outline through the necessary self-guided learning and engineering design required to meet the contributions.

A2.1 Case Study 1: AMME5520 Advanced Control and Optimisation

A2.1.1 LO1. *Approach research papers in a professional and research-orientated manner, and conduct critical reviews of these papers*

This learning objective is met through the thesis literature review and gap synthesis across UAV DAA radar pipelines, GPU radar processing, monopulse angle estimation under low SNR, and multi-target tracking. The thesis requires evaluating assumptions, identifying limitations, and justifying design choices based on evidence, aligning with Week 13 Lightning talks (LO1) and the unit's self-directed critical engagement with research.

A2.1.2 LO2. *Implement simple path generation algorithms, controllers and decision metrics for an autonomous system in order to meet specific mission objectives*

This learning objective is met by implementing autonomy-relevant estimation and decision components in the detection-to-track chain: measurement formation, data association (e.g., NN/GNN/JPDA variants), tracking filters (EKF/UKF/IMM-style variants), and track lifetime management. Although the thesis project does not explicitly control the position of the targets that it is making estimations on, estimation is an essential component of any control system, and multi target tracking engages with the state based decision making that this learning objective is concerned with. The estimation theory in this project has heavy alignment with Weeks 07–08 in particular (Kalman filtering and extensions).

A2.1.3 LO3. *Understand a number of different path generation and control algorithms implemented in autonomous systems and how they are linked to optimality criteria, platform stability and vehicle constraints.*

Tracking aerial vehicles that can both fly in a straight line and conduct several manoeuvres motivates the use of multiple filters and tracks that model possible trajectories of the targets. Although targets are not being controlled, multi target tracking itself is an optimisation problem. Understanding stability is essential to minimise track id switching when multiple targets are within a few standard deviations of one another (in terms of state).

This learning objective is also met through the comparative evaluation of tracking algorithms, where we optimise for high track quality and low computational resource use. This aligns with Weeks 02–03 (optimality-driven algorithms), Weeks 07–09 (estimation, uncertainty, robustness).

A2.1.4 LO4. *Understand how cost functions are used to define mission objectives in a mathematical form, so that autonomous systems can make decisions about their next action.*

This learning objective is met by framing pipeline design as an objective-driven trade-off between sensitivity/track quality and computational cost, using ablations and parameter sweeps to identify design points. This aligns with Week 01 (optimal control framing), Weeks 10–11 (optimisation/MPC), and the unit's focus on formal objective definition. Additionally, track association is an optimisation problem.

A2.1.5 Coverage statement

The thesis addresses LO1–LO4 through critical review, implementation of estimation/decision algorithms, comparative evaluation under constraints, and objective-driven trade-off analysis, demonstrating at least 75% coverage.

A2.2 Case Study 2: ELEC3305 Digital Signal Processing

A2.2.1 LO1. *Describe signals using mathematical concepts such as convolutions, transforms, spectral analyses, linear difference equations, filters, correlation and covariance*

This learning objective is addressed through frequency-domain radar processing. Convolutions will be used in the form of matched filters in the dechirping process. The Fourier transform will be applied with the FFT algorithm over both fast time and slow time to produce range and Doppler maps. Hann, Hamming, and other filters will be applied to minimise spectral leakage, and the spectral effect of our hardware configuration and geometry will be compensated for. Correlation, linear difference, and covariance will be essential concepts in the process of detection and tracking of targets. This aligns strongly with Week 03 (convolution and frequency response), Weeks 07–10 (DFT/FFT properties and spectral analysis), and the unit's emphasis on transform-based signal description.

A2.2.2 LO2. *Use software, such as MATLAB and Python, to develop code to analyse and process signals including filtering, inverse filtering, spectral analyses, resampling, time-frequency analyses*

This learning objective is addressed through substantial software implementation and testing of DSP stages in an end-to-end pipeline, including benchmarking CPU vs GPU implementations and developing validation/profiling tools with parameter sweeps. This aligns with Weeks 02–12 practicals, which focus on implementing DSP in software. Python and C++ will be used to develop both the pipeline and the simulation application.

A2.2.3 LO3. *Identify linear signal models for time-invariant systems such as convolutional models, sinusoidal and harmonic models, echo and reverberation models, modulation models and time-frequency models.*

This learning objective is addressed by using standard linear and time-frequency models for radar returns and mapping sampled signals into range–Doppler representations, then studying how model behaviour changes under low SNR and limited resolution. This aligns with Week 02 (signal models), Weeks 04–06 (DTFT/Z-transform/transfer functions), and Weeks 11–12 (sampling and ADC/DAC considerations).

A2.2.4 LO4. *Design and evaluate signal processing systems.*

This learning objective is addressed directly by designing and evaluating alternative pipeline configurations (windowing, FFT sizing/batching, CFAR variants, integration policies, and angle sensing), and quantifying effects on detection sensitivity, false alarms, and end-to-end latency/throughput. This aligns with Weeks 04–10 and the unit's system design emphasis.

A2.2.5 Coverage statement

The thesis covers LO1–LO4 through transform-based processing, software implementation, linear/time-frequency modelling, and end-to-end system design and evaluation, demonstrating at least 75% coverage.

A3 Work Health and Safety Report

TODO: Write the health and safety report

TODO: I don't quite know what this involves, but it is worth 5% so I shouldn't ignore it

A3.1 Work, Health and Safety at Mission Systems

A3.2 Incidents and Preventions During Placement

A4 Practical Experience Logbook